NanoBook: Final Report

Every test, why we ran it, what it told us · C++23, GCC 14 / Clang 19, Linux · 25 October 2026

1. Summary

NanoBook is an order book: the data structure at the heart of every stock exchange and every trading firm. It was built in seven phases, and each phase ran tests to check two things: **is it right**, and **is it fast**. This report goes through every one of those tests in order. For each it says, in plain words:

1. **The test:** what we ran.
2. **Why we expected it to win:** the reasoning before the data.
3. **What it told us:** the numbers, and whether the guess was right.
4. **So what:** what it means for NanoBook, and for anyone building one.

It ends with a conclusion. The phase reports (`docs/phase1-report.md` to `docs/phase6-report.md`, `docs/bare-metal-report.md`) have the full detail; this report is the story.

What it is	A limit order book and matching engine in C++23, plus a feed handler that rebuilds every stock's book from a real Nasdaq day
Correct?	The fast book agrees with a deliberately simple reference book on 162 million random operations and after each of 363 million real order messages. Every planted bug (over 75 across the phases) was caught
Fast?	A real Nasdaq day (368 million messages) at **about 68 ns per message** on one core; the slowest 1 in 10,000 messages at **about 0.61 µs** (bare metal, isolated core, release commit `f895be6`)
Pipeline	Three threads over lock-free ring buffers: **0.63–0.69 µs at p99** from feed to publisher, up to 4 million messages a second
Experiments	11 optimisation ideas tested one at a time: 4 kept, 7 not, each with its data

2. The problem, in plain words

2.1 An order book is a queue at a bakery, sorted by price

Picture a bakery that sells one kind of loaf, where buyers shout the price they'll pay and sellers the price they'll accept:

```
SELLERS (asks)   $10.03  [Carol: 500]
                 $10.02  [Ann: 100] [Bob: 200]    <- best ask: the cheapest seller
BUYERS  (bids)   $10.01  [Dan: 400]               <- best bid: the highest buyer
                 $10.00  [Eve: 300] [Fay: 400]
```

Each price is a **level**, with its own first-come-first-served queue. When a new buyer offers $10.02 for 250 loaves, they trade with Ann first (she was there first), then 150 of Bob's. That rule, best price first and then oldest first, is **price-time priority**, and the program that applies it is a **matching engine**.

2.2 A trading firm keeps its own copy of the exchange's book

Nasdaq doesn't send a picture of its books. It sends only the **changes**, one message each, in a feed called **ITCH**: "order 501 added: buy 100 at $10.01", "order 501: 30 executed", "order 501 deleted". A firm applies each change to its own copy, and its copy then matches Nasdaq's at every moment. A real day is about 368 million such messages.

2.3 Why nanoseconds matter

A firm that sees a price change 1 µs before its competitors can act on it first. So the book must apply each message fast, and, more important, **never be slow**: one message that takes 50 µs holds up every message behind it. That's why this report talks about **percentiles**:

- **p50** (the median): half the messages are faster than this. The typical case.
- **p99**: 99 in 100 are faster. The slowest 1%, called the **tail**.
- **p99.99**: the slowest 1 in 10,000.

A system with a fast average and a slow tail is a bad trading system.

2.4 Why cache matters more than "Big-O"

The CPU works on data in its caches: small, fast memories next to it. Reading from the nearest cache takes about 1 ns; from main memory, about 100 ns. Data moves in **cache lines** of 64 bytes. So a structure whose data sits side by side in one array (read one line, get eight neighbours) beats one whose pieces are scattered across memory (one trip to main memory each), even when textbook analysis ("O(log n)") says they're equal. Most of NanoBook's design follows from this.

---

3. Phase 1: the reference book, the answer key

The plan: write a slow, obviously correct order book first, using the standard library's `std::map` (a sorted tree) for price levels and `std::list` for each queue. Every faster book afterwards is checked against it.

3.1 Unit tests for every order type

- **The test:** 109 tests, each a small story with a known ending. "Add a buy of 100 at $10; add a sell of 30 at $10: expect a trade of 30, and 70 left resting." Every order type (limit, market, immediate-or-cancel, fill-or-kill, cancel, modify) and every edge (empty book, partial fills, an order trading with its own owner).
- **Why:** the reference book is the answer key for everything after it. If it's wrong, every later comparison is meaningless.
- **What it told us:** all pass on GCC and Clang, with and without sanitizers (tools that catch memory errors and undefined behaviour as the tests run). Writing them found design gaps, for example the exact order of checks when an order is invalid in two ways at once.
- **So what:** the behaviour is pinned down in writing (`docs/design.md`) and in tests, so "the same as the reference book" has a precise meaning.

3.2 Invariants, checked after every input

- **The test:** after every single input in every test, check seven rules that must always hold: the book isn't crossed (best bid below best ask), no empty levels, each level's total equals the sum of its orders, every order is in exactly one level, queues are in arrival order, filled plus remaining equals total.
- **Why:** a test checks the ending it was written for. An invariant checks everything, all the time, including states nobody thought to write a test for.
- **What it told us:** they hold everywhere.
- **So what:** the same seven rules became the structure check used on the real day (§6.2) and in the fuzzers.

3.3 Planted bugs: do the tests have teeth?

- **The test:** put 13 bugs in on purpose (match the newest order first; let an IOC remainder rest; ignore the owner when checking self-trades...), run the tests, then restore the code.
- **Why:** a test suite that passes proves nothing unless it would fail on broken code.
- **What it told us:** all 13 were caught. One was caught by only one test, which showed a gap; a test was added.
- **So what:** every phase after this planted bugs in its own new code (over 75 in all). It became the standard check that a test suite works.

3.4 The first benchmark: where the time goes in a tree-based book

- **The test:** time each operation (Google Benchmark, one pinned core).
- **Why:** to have a baseline that later phases must beat.
- **What it told us:** adding an order made **2.01 heap allocations** (a request to the memory manager for new memory, slow and unpredictable). Cancel was the slowest operation at about 137 ns: it walks two trees, about 14 pointer hops for 10,000 orders, each possibly a trip to main memory.
- **So what:** the target for phase 2: no allocation while trading, and no pointer chasing.

---

4. Phase 2: the fast book

The plan: same behaviour, built for speed: every order slot made at start-up in one array, a flat hash table to find orders by id, and price levels in a sorted array.

4.1 The zero-allocation test

- **The test:** replace the program's memory allocator with one that counts calls, warm up, run 100,000 random operations on the fast book, and require exactly **0** allocations.
- **Why:** allocation is slow and unpredictable (sometimes the memory manager has to ask the operating system for more), so it's banned while trading (plan rule 1). A rule needs a test, or it erodes.
- **What it told us:** 2.01 allocations per add (reference) → **0**. The test runs on every commit.
- **So what:** the fast book's speed doesn't depend on the memory manager's mood.

4.2 The differential fuzzer

- **The test:** generate random streams of orders (seeded, so any failure can be replayed), feed each one to the reference book and the fast book, and compare every event and the whole book after every operation.
- **Why:** the two books share no code. If two independent programs agree on millions of random inputs, both are very likely right. Random inputs reach situations nobody would write a test for.
- **What it told us:** **162 million** operations, 0 differences. All 12 bugs planted in phase 2 were caught by the unit tests or the fuzzer.
- **So what:** "correct" for the fast book means "identical to the answer key", checked at scale. Every later change to the fast book was re-checked this way.

4.3 Orders in a pool, not in list nodes

- **The test:** benchmark the fast book with its order pool against the reference book.
- **Why we expected it to win:** a `std::list` node is a separate allocation, placed wherever the memory manager had room. A pool keeps every order in one array, and links them with 4-byte slot numbers instead of 8-byte pointers.
- **What it told us:** add **3.0×**, cancel **2.4×**, match **2.5×** faster.
- **So what:** where data lives matters as much as what the code does.

4.4 Finding an order by id: a flat hash table

- **The test:** replace a temporary `std::unordered_map` (the standard hash map) with `IdTable`, a flat table: linear probing (a collision just tries the next slot), sized at start-up.
- **Why we expected it to win:** `std::unordered_map` allocates a node per entry and follows a pointer to it; a flat table usually finds the entry in the first cache line it reads.
- **What it told us:** add **2.1×**, cancel **1.4×** faster, and the last allocation gone.
- **So what:** the standard containers are general; a structure that knows its size and its keys can be much faster.

4.5 Price levels: tree, sorted array, or one slot per price?

- **The test:** three level structures, same benchmarks, one session: `std::map` (a tree), a sorted array with the best price at the end, and a tick array (one slot for every possible price, with a bitmap to find the next non-empty one).
- **Why we expected what:** the sorted array keeps levels side by side, and most activity is at the best price, which sits at the end, so inserting there moves nothing. The tick array finds any price in one step.
- **What it told us:** the sorted array beat the tree on sweeps (1.7–1.9×) and removed allocations. The tick array was up to **3.9× faster** still on cancels in a large book.
- **So what:** the tick array was faster but not chosen, because its memory grows with the **width of the price range**, not the number of levels. Phase 4 later showed how fatal that is on real data (§6.4). The sorted array works for any price.

4.6 The hybrid search: an "obvious" win that lost

- **The test:** before the binary search in the sorted array, check the 4 levels nearest the best price one by one.
- **Why we expected it to win:** most orders arrive near the best price.
- **What it told us: slower on every benchmark.** Dropped.
- **So what:** "obviously faster" was wrong. (Phase 6 tried it again on real data, and it won by 10%: the synthetic benchmark's orders were spread more evenly than real ones. §8.3.)

4.7 Order slot: 48 bytes, or padded to a whole 64-byte cache line?

- **The test:** every benchmark with the order slot at 48 bytes (packed) and at 64 (padded).
- **Why we expected 64 might win:** at 48 bytes, half the orders straddle two cache lines, so touching one may load two.
- **What it told us:** no case for 64: every difference within ±6%, most inside the noise.
- **So what:** keep 48 (a quarter less memory). The benchmark's 4.8 MB of orders fits in the 30 MB cache anyway, where a second line is cheap.

4.8 A weakness found by accident: deep inserts

- **The test:** none planned. A benchmark setup that built 10,000 levels worst-price-first took minutes.
- **What it told us:** inserting a new price deep in a sorted array shifts every better level by one: 5,000 levels × 32 bytes = 160 KB moved for one order.
- **So what:** a known tail-latency risk, written down to measure on real data (phase 4: rare, §6.5; phase 6: closed, §8.8).

---

5. Phase 3: measuring without fooling ourselves

The plan: a timing harness that measures every single operation, not averages: read the CPU's time-stamp counter (`rdtsc`) before and after each operation, and put each result in a histogram.

5.1 How much does the stopwatch itself cost?

- **The test:** time an empty region millions of times.
- **Why:** reading the counter takes time too, and at 15 ns per operation that matters.
- **What it told us:** about **11.6 ns** minimum on this CPU, now subtracted from every sample. The counter ticks 2.4192 times per ns, and drifts under 0.03 parts per million during a run.
- **So what:** the harness can resolve operations of a few nanoseconds honestly.

5.2 The first full latency run, on WSL2

- **The test:** every scenario on all three books, 1,000,000 timed operations each, on WSL2 (Linux inside a Windows virtual machine, this laptop's everyday setup).
- **What it told us:** FastBook's add p50 about 20 ns, cancel about 49 ns. But every scenario, even the simplest, had a p99.99 of about **2.3 µs**, the same in every test.

- **So what:** a floor that doesn't depend on what the code does is the machine, not the code. These numbers were marked **NOT FOR PUBLISHING** until bare metal could confirm it (§9.1). It did.

5.3 Two tools disagreeing

- **The test:** the harness's throughput mode against Google Benchmark, same operation (match).
- **What it told us:** 9.3 ns against 13.1 ns, unexplained on WSL2.
- **So what:** written down as an open question rather than averaged away. On bare metal both gave 8.7 ns (§9.4): a virtual-machine effect.

---

6. Phase 4: a real Nasdaq day

The plan: read one full real trading day (30 January 2019, 11.2 GB, 368 million ITCH messages) and rebuild every stock's book from it.

6.1 Parser tests and a golden replay

- **The test:** decode hand-written ITCH bytes (written from the spec's tables, not by our own encoder) and compare every field; replay a small made-up day (written by our test encoder, so no Nasdaq data sits in the repo) and compare its final books with a recorded answer.
- **Why:** the parser turns raw bytes into fields. An offset wrong by one byte gives plausible nonsense, not a crash.
- **What it told us:** all pass, and the golden replay runs on every commit.
- **So what:** a parser bug can't slip in unnoticed.

6.2 The whole day, checked

- **The test:** replay all 368 million messages and account for every one: every order message must apply cleanly (no unknown order, no over-reduction, no duplicate, no full book); no book may be crossed while its stock is trading normally.
- **Why:** this is the correctness test only real data can give.
- **What it told us: 363,118,215 order messages, all accepted.** 8 crossed books, every one while the stock was paused or quotation-only, when crossing is allowed.
- **So what:** our copy follows Nasdaq's book exactly, all day, for all 8,713 stocks.

6.3 The whole day, fast book against reference book

- **The test:** replay the day into both books at once; compare them quickly after every order message, and in full (every level, every queue, every order) every million messages and at each market event.
- **Why:** random tests (§4.2) can miss what real flow does.
- **What it told us: 0 differences** in 363 million quick checks and 61,354 full ones.
- **So what:** the strongest evidence in the project that the fast book is right.

6.4 Could the tick array hold a real day?

- **The test:** size a tick array for every stock's real price range.
- **Why:** it was the fastest structure in phase 2.
- **What it told us: 489,000 GB.** Quotes parked far from the market ("stub quotes", placed where they will never trade) stretch the price range of many stocks enormously.
- **So what:** the sorted array was the right default; a structure that's fastest in a benchmark can be impossible in reality.

6.5 What real flow looks like

- **The test:** count, over the whole day, how far new orders land from the best price and how many levels each new level shifts in the sorted array.

- **What it told us:** 93% of new orders land within $1 of the best price; only **0.02%** of new levels shift 1,024 or more levels.
- **So what:** deep inserts (§4.8) are real but rare. And real orders cluster at the best price, which reopened the hybrid search (§8.3).

6.6 Timing the real day

- **The test:** replay the day with every 8th order message timed (WSL2 at first).
- **What it told us:** 104 ns per message, of which the parser was **3%** and the book 97%; p50 105 ns, p99 429 ns, p99.99 about 11 µs. The same book runs **2.5–5× slower** per message than in phase 3's benchmarks.
- **So what:** in a benchmark the book fits in cache; on the real day 8,713 books don't. Cache misses, not instructions, set the speed. That became phase 6's main lead.

---

7. Phase 5: three threads and lock-free rings

The plan: split the work across three cores: a **feed** that reads and parses, an **engine** that keeps the books, a **publisher** that logs each update. They hand records to each other through ring buffers NanoBook writes itself, made safe by C++ memory ordering, with no locks.

7.1 ThreadSanitizer, and a planted ordering bug

- **The test:** run every test under ThreadSanitizer (a tool that watches every memory access between threads), and plant a bug: publish the "record ready" counter with `relaxed` ordering instead of `release`.
- **Why:** with `relaxed`, the CPU or compiler may make the counter visible before the record itself. On x86 it almost never shows, so ordinary tests pass.
- **What it told us: every ordinary test passed with the bug in; ThreadSanitizer caught it.**
- **So what:** concurrency bugs need tools, not just tests. TSan runs in CI on every push.

7.2 The same answer as one thread

- **The test:** run the whole day through three threads and through one, logging every book update to a binary file, and compare the files.
- **What it told us: byte-identical**: 363,118,215 updates, 20.3 GB, the same checksum.
- **So what:** the pipeline changes nothing about the answer, only who computes it.

7.3 How fast is a hand-over between cores?

- **The test:** one ring between two pinned cores: stream records, and bounce one back and forth.
- **What it told us:** a round trip (there and back) takes about 155–175 ns at p50 on bare metal.
- **So what:** moving one cache line between cores costs tens of nanoseconds: as much as a whole book operation.

7.4 Latency under load, without coordinated omission

- **The test:** pace the feed at a fixed rate (0.5 to 8 million records a second), and stamp each record with the time it was **due**, not the time it was sent. Measure to the publisher.
- **Why the stamping matters:** if the feed is held up for 50 µs and stamps records when they finally leave, the 50 µs wait never appears. That mistake is called **coordinated omission**.
- **What it told us (WSL2):** p50 about 500 ns up to 2 M/s, saturation at 4.4 M/s, and a p99 of **65–72 µs at every rate**, even the lowest.
- **So what:** a tail that doesn't grow with load isn't queueing. It was the virtual machine (§9.1).

7.5 Are three threads faster than one?

- **The test:** the whole day, unpaced, one thread against three.

- **Why we didn't expect much:** the book is 97% of the work, so the engine is the bottleneck either way.
- **What it told us:** three threads were **slower**: 0.69–0.78× on WSL2, 0.80× on bare metal (0.78× at the release commit).
- **So what:** a pipeline isn't free; each hand-over moves cache lines between cores. It pays only when it takes real work off the critical thread (writing the 20 GB log to disk, three threads won: 81.7 s against 86.0 s).

---

8. Phase 6: eleven experiments

The method: each idea starts from a measured symptom, states a hypothesis and the number that should move, makes the smallest change that tests it, and is timed against the old code in the same session, alternating before and after (ABBA order, so a machine that warms up hits both equally). It's kept only if it passes a rule written down before the timing.

8.1 E1: the feed hammering a full ring

- **Symptom:** three threads slower than one (§7.5); ring 1 was found full 4.5 times per message.
- **Hypothesis:** each time, the feed re-reads the engine's counter, pulling its cache line away from the engine, which must pull it back.
- **The test:** (a) the engine publishes its counter every 64 records; (b) the feed waits 64 `pause` instructions after finding the ring full; (a)+(b).
- **What it told us:** all three about **15% faster** on three threads, tied. Kept (b), the simplest. Bare metal confirmed it: 16% faster, with the one-thread control unchanged.
- **So what:** the cheapest fix for contention can be to look less often.

8.2 E2: page faults in the level arrays

- **Symptom:** a p99.99 of about 11 µs on the real day.
- **Hypothesis:** the level arrays are only reserved, so the first touch of each 4 KiB page while trading is a page fault (the operating system stops the program to map it).
- **The test:** touch every page at start-up.
- **What it told us:** faults while timed 4,758 → **0**, p99.99 about 12% better on WSL2. On bare metal, faults 1,650 → 0 but p99.99 moved only 2–6 ns, inside the noise: a virtual machine makes each fault much more expensive. A re-test on today's code (with huge pages) found no difference either way, 1 fault without the touching.
- **So what:** by its rule, remove it. **Kept anyway, by decision:** that 1 fault was luck (the untouched arrays shared pages with touched ones), and touching makes 0 faults a guarantee at no measured cost. A rule can be overridden, but only with the reason written down.

8.3 E3: the hybrid search, second try

- **Symptom:** real flow clusters at the best price (§6.5), unlike phase 2's benchmark.
- **The test:** check the K levels nearest the best before binary search, K = 4, 8, 16.
- **What it told us:** K = 4: **10% faster on the real day** (11% on bare metal). K = 8 and 16 slower on adds.
- **So what:** the same idea lost on made-up data and won on real data. Test on the workload you actually have.

8.4 E4: compiler options

- **The test:** link-time optimisation (LTO), `-march=native` (use every instruction this CPU has), profile-guided optimisation (PGO: compile, run on real data, recompile using the profile).
- **Why we expected wins:** all three are standard advice.

- **What it told us:** LTO: no gain (the hot code is all in headers, already compiled together). `-march=native`: **1.8% slower** (the parser got slower). PGO: 1.4% more throughput, but replace messages' p50 **25% worse**.
- **So what:** none became the default. Standard advice is a hypothesis, not a result.

8.5 E5: ring padding, 64 or 128 bytes

- **The test:** the ring's two counters 64 bytes apart instead of 128.
- **Why 128 in the first place:** Intel's prefetcher tends to pull cache lines in pairs, so two counters 64 bytes apart can still interfere.
- **What it told us:** 64 bytes made the round trip worse in 4 of 4 runs. 128 stays.
- **So what:** "a cache line is 64 bytes" isn't the whole story on this CPU.

8.6 E6: huge pages

- **Symptom:** the book is 2.5–5× slower on the real day than in cache (§6.6).
- **Hypothesis:** besides cache misses, address translation: the books' 400 MB is about 100,000 normal 4 KiB pages, far more than the CPU's translation cache holds.
- **The test:** put the books' arrays in one memory arena on 2 MiB pages.
- **What it told us: 5.6% more throughput, p99 11% better** (WSL2); **7%** on bare metal.
- **So what:** kept. Memory layout reached beyond the program, into how the operating system maps it.

8.7 A profile, and E9: the ID table

- **The test:** sample where the real day's time goes (`perf`), attributed to source lines.
- **What it told us: half the replay was in the ID table**, mostly one line (comparing ids); the levels about a quarter.
- **E9a, prefetch the next message's ID entry:** 6% slower; reverted.
- **E9b, halve the ID-table entry (16 → 8 bytes):** keep a 32-bit tag from the hash instead of the whole id, and confirm a match from the order itself. On WSL2 the result sat inside the noise and was reverted pending bare metal; there it was **7% faster in all 4 rounds, p50 24% better**, and re-added.
- **So what:** profile before redesigning: it moved the main target. And a noisy machine can hide a real gain as easily as invent a fake one.

8.8 E7: the moving window, and E8: deep inserts

- **E7, the test:** a tick array for the 1,024 prices nearest the best (one-step lookup there), the sorted array beyond. Built, and proven identical to the reference book on the whole day.
- **What it told us:** 7–13% faster on average on WSL2 (15% on bare metal), but the tail worse in every round: on bare metal p99 equal and p99.99 **2.5–6× worse**. Two fixes didn't change it.
- **So what:** not the default; it stays a tested option. A trading system is judged on its tail.
- **E8, deep inserts:** closed with arithmetic instead of an experiment: at most 0.16% of the replay, smaller than the noise of any timing.

8.9 E1d: the same trick on the second ring

- **The test:** batch the engine's updates to ring 2 (three variants).
- **What it told us:** three threads 7–12% faster unpaced, but paced latency worse (p90): updates waited for their batch.
- **So what:** not kept. Batching was free where it delayed free slots (E1) and costly where it delayed the output.

---

9. The bare-metal session: checking against the real machine

Every number above marked WSL2 came from a virtual machine. One session booted the laptop straight into Linux from a USB stick, with three cores reserved for the measurements (`isolcpus`, `nohz_full`), and re-ran everything.

9.1 Were the slow tails the code or the machine?

- **What it told us:** per-operation p99.99 fell **5–50×** (the 2.3 µs floor vanished); the real day's p99.99 from about 11 µs to **0.65 µs** (the trusted round); the pipeline's p99 from 65–72 µs to **0.65 µs**, still flat up to 4 M/s. p50 and p99 barely moved.
- **So what:** the code was measured correctly on WSL2 in the middle of the distribution, and the tails were the virtual machine. Publishing WSL2 tails would have understated NanoBook by 100×.

9.2 Re-checking every kept change

- **What it told us:** E1b (16%), E3 (11%) and E6 (7%) **confirmed**; E2 failed its rule (§8.2); E9b passed (§8.7); the window's tail moved further out but stayed (§8.8).
- **So what:** a virtual machine can flatter or hide a change. Decisions made there need checking where it counts.

9.3 What the hardware counters say

- **What it told us:** per real-day message, about 377 cycles, **0.52 trips to main memory** and 0.05 page walks: cache misses, not address translation, dominate. Across threads, ring 1's cache line moved about 0.1 times per message after E1b.
- **So what:** the hypotheses from phases 4–6 were tested with the CPU's own counters, which WSL2 can't read.

9.4 The tool disagreement, solved

- **What it told us:** FastBook's match at 8.7 ns in both tools (§5.3).
- **So what:** a virtual-machine artifact, now explained instead of guessed. (E9b, re-added after this session, later made this synthetic `match` slower: §10.6.)

---

10. Phase 7: making it checkable

The plan: turn the work into something a stranger can check in minutes: one command that reproduces the numbers, a front page with the results, and every published number measured again at one commit, the release.

10.1 libFuzzer: random bytes into the feed handler

- **The test:** a fuzzer that feeds millions of byte strings to the ITCH parser and on into two book builders at once, one over each book. It fails on a crash, a memory error, a broken book, or the two books disagreeing. Two modes: raw bytes, and a list of small operations that the test encoder writes as valid ITCH.
- **Why:** a feed handler reads bytes it doesn't control. Unit tests check the bad inputs someone thought of; a fuzzer looks for the ones nobody did.
- **What it told us:** raw bytes alone reached 179 branches in 60 s and never built a book (random bytes almost never make a valid stock followed by an order for it). With both modes: 915 branches. A planted bug (a level's total not reduced) was caught in **0.4 s**; clean runs of 60 s and 120 s found nothing. It now runs for 60 s in CI on every push.
- **So what:** a fuzzer needs help to get past the first check. Valid-looking input with nonsense inside (unknown stocks, deletes of orders that never existed, full books) is what tests the code behind the parser.

10.2 The stranger test

- **The test:** `scripts/reproduce.sh` builds, runs every test, fuzzes 30 million operations and prints the latency table next to the published one. On the USB stick, a clean Ubuntu, the repository was cloned into an empty folder and the script run from nothing.
- **Why:** phase 7's exit criterion is "a stranger can build, test and reproduce the numbers in under 10 minutes". That has to be timed, not assumed.
- **What it told us: 55 s** from the fresh clone to the table, the GoogleTest and Google Benchmark downloads included: 790 tests passed, 3 × 10 million fuzzed operations PASS. The full level, with the real day and the pipeline, took 393 s on WSL2, plus the day's 4.76 GB download.
- **So what:** the criterion is met, with plenty of room. The download is the reader's internet, not ours, so the README states it separately instead of hiding it inside "10 minutes".

10.3 The release session: every number at one commit

- **The test:** one more bare-metal session, at the release commit `f895be6`, re-running every published number: phase 3's operations, the real day twice, the pipeline at every rate.
- **Why:** the first bare-metal session (§9) ran before E9b was re-added, so the code had moved on. A front page that mixes commits describes code that no longer exists.
- **What it told us:** the real day **74.7–75.4 → 67.7–68.0 ns per message**, still 0 page faults while timed. Its p99.99 was **0.61 µs in both rounds**; the first session had 0.65 µs in one round and 4.0 µs in the other, a round not trusted because commands ran on other cores during it. The pipeline's p99 stayed at 0.63–0.69 µs up to 4 M messages a second. In the synthetic tests, p50 and p99 barely moved except `match` (§10.6), but p99.99 was **1.3–7× lower** for every operation (add: 299–318 → 43–44 ns). E9b changed only the ID table, so it can't explain that. A possible cause, not tested: the first session ran its scripts one at a time, with other work between them and, at least once, during one (phase 4's round 1); the release session was one script, left alone.
- **So what:** every number in the README comes from one session, one commit and one machine, each stated next to it. And a p99.99 from one session is a sample of the machine as well as the code: two sessions on the same laptop disagreed by up to 7× there, while their p50s (`match` aside) stayed within 2.5 ns.

10.4 What phase 6 bought, in total

- **The test:** the end of phase 5 (`7d557fa`) against the release, on the real day, two rounds, both builds from commits.
- **Why:** each experiment in §8 was measured against the code just before it. This answers the question an interviewer actually asks: "how much faster did all of it make it?"
- **What it told us: 86.4 → 67.9 ns per message** in both rounds: 21% less time per message. p50 90 → 62 ns (31% better), p99 346–356 → 295–296 ns, p99.99 705–712 → 606 ns. Executions gained most: their p50 halved, 88 → 43 ns.
- **So what:** the four kept changes improved the tail as well as the average. That was the rule for keeping them, and it's why the moving window wasn't kept (§8.8).

10.5 The flame graph: where the time goes

- **The test:** sample the real-day replay's call stacks (`perf`, 94,193 samples) and draw them as a flame graph: each function a bar, as wide as its share of the time.
- **Why it took care:** the obvious build for profiling is `-O2`, which is 2.4× slower here, a different program. So the graph profiles the release build plus debug information, and the script checks that the instructions are identical (0 of 50,118 lines differ). The stacks come from the CPU's own record of calls (LBR), because GCC gives the hottest function, a leaf, no frame pointer.
- **What it told us:** the graph covers the whole program. 41% is the untimed pre-scan that sizes the books, 48.5% is the timed replay. Inside the replay, the **ID table takes 60%** of the time (its lookup alone 50%) and the price levels 24%.
- **So what:** it confirms phase 6's profile with a different tool. The next speed-up would have to come from the ID table's memory traffic. And a profile needs a caption: without one, a reader would think the pre-scan's `std::unordered_map` was in the hot path.

10.6 A cost found by the release session: `match` since E9b

- **The test:** the release session's synthetic `match` (one order fully fills one resting order) against the first session's. Then, on WSL2: the two commits' machine code, their instruction counts (Valgrind's cachegrind), and an A/B of both builds.
- **Why:** E9b was the only hot-path change between the two commits, and it made the real day 7% faster. A slower micro-benchmark needed explaining, not hiding.
- **What it told us:** on bare metal `match` got **8–17% slower** (harness p50 14.9–15.3 → 16.9–17.4 ns). The WSL2 A/B agreed: 11% slower at p50, every round. But the work hardly changed: 10 more instructions per match (+2%), and no extra branch misses. The likely cause, still a hypothesis: confirming a tag match reads the order's id from the pool, a second read that has to wait for the first. In a tiny book that sits in L1, E9b's gain (twice as many entries per cache line) can't show, and only that wait remains. On the real day matches got **13–21% faster** at p50.
- **So what:** published as measured, with one line on the front page. It's the hybrid search (§4.6, §8.3) in reverse: a micro-benchmark and the real workload can disagree in either direction. A fix is noted but not built: every caller already knows the order's slot, so an erase could compare slots and skip the extra read.

10.7 The documents, checked

- **The test:** every link, image and repository path in every Markdown file, checked against the repository; every design note's status header read against what happened.
- **What it told us:** four stale paths (files planned but built elsewhere, or later removed), and four status headers that still said bare metal was pending.
- **So what:** the documents are part of what's shipped. A reader who follows a dead link stops trusting the numbers next to it.

---

11. Conclusion

What NanoBook is

A limit order book and matching engine in C++23, with a feed handler that rebuilds every stock's book from a real Nasdaq day. A simple reference book is the answer key; a fast book does the work; a three-thread pipeline over its own lock-free rings moves the data.

How fast

On one isolated core of a laptop CPU: **68 ns per message** over a whole real day, 368 million messages, with half of them under 63 ns and the slowest 1 in 10,000 at 0.61 µs. Single operations in a book that fits in cache: 17–51 ns at p50. From feed to publisher through three threads: 0.63–0.69 µs at p99, up to 4 million messages a second. No heap allocation and no page fault while trading.

How correct

The fast book produced the same events and the same book as the reference on 162 million random operations, and after every one of the day's 363 million order messages. 790 tests run in six builds, under AddressSanitizer, UndefinedBehaviorSanitizer and ThreadSanitizer. Over 75 bugs were planted on purpose to check that the tests catch them; every one was caught.

What was learned

1. **Where data lives decides the speed.** Every large gain came from memory layout: one array of orders instead of list nodes (2.4–3.0×), a flat ID table (1.4–2.1×), 8-byte entries (7%), huge pages (7%). Instruction counts barely mattered.
2. **Test on the workload you have.** The fastest structure in a benchmark (the tick array) was impossible on real data. The hybrid search lost on made-up orders and won on real ones; E9b won on the real day and lost a micro-benchmark.

3. **The machine is part of the measurement.** The virtual machine's tails were 100× too slow, and its noise hid a real 7% gain. Published numbers need bare metal and isolated cores.
4. **Judge by the tail.** Three ideas made the average faster and the slowest messages slower. A rule written before the timing stopped each one from being kept.
5. **Standard advice is a hypothesis.** `-march=native`, LTO and PGO, all textbook, gave nothing or cost something here.
6. **A pipeline isn't free.** Three threads ran at 0.78× the speed of one: each hand-over moves cache lines between cores. Threads pay only when they take slow work off the critical path.

What would come next

1. Find why the moving window's tail is slow (cache-miss sampling per message kind), then reconsider it.
2. Shard the 8,713 books across several engine threads, by stock.
3. Read the feed from UDP multicast (MoldUDP64), the way ITCH is really delivered, instead of a file.